

Fig. 7: Python Shell window

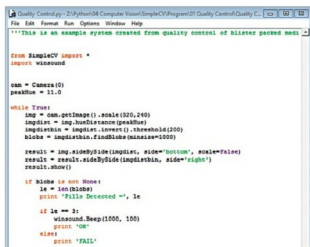


Fig. 8: Python code

Control.py file as shown in Fig. 8. Copy the code from this file or use the file as it is, because Python is an indentation-sensitive language.

## Import modules

Modules required in the program are:

1. from SimpleCV import \* # importing everything from SimpleCV
2. import winsound # importing winsound to play sound

The winsound module comes prepackaged with Python, while we have already installed SimpleCV using aforementioned steps.

## Functions used

**Initialise camera.** You need to be sure that your camera is detected as 0 or 1 and update the program accordingly as given below:

```
Cam = Camera(0) # initialise the camera
```

**Peak hue level.** Enter peak hue level of the colour to be detected. In this project, tablets used had the peak hue of 11.0. You need to find it out for your tablets and update the same in the code as given below:

```
peakHue = 11.0
```

The software counts the number

of tablets by detecting the objects matching the pre-defined colour and separating them from the rest of the scene as shown in Fig. 11. Here we have used an HSV system instead of RGB as it performs better in such applications.

To find out the hue value for the tablets you are using, run the following code:

```
from SimpleCV
import *
cam = Camera(0)
while True:
```

```
img = cam.getImage()
peak = img.huePeaks()
print peak
```

A separate hue.py Python file is also provided for hue. Now, put the tablet pack under the camera and run this code. Ensure optimal lighting as excessive light will reflect off the silver background of tablets to create false results. When you run the code, you will start to see many numbers. Just note the first one in the list. That is the hue value for your tablets.

**Capture image.** The following function captures the image (Fig. 9) of the tablet pack and scales it down to 320x240 resolution:

```
img = cam.getImage().scale(320,240)
```

**Hue distance image.** The function below creates another image that indicates the hue distance for the value you selected:

```
imgdist = img.hueDistance(peakHue)
```

The tablets should look black (Fig. 10) because the difference is zero there. If not, you have not selected the right hue value.

**Inverted image.** The tablets will become white (Fig. 11) after this command:

```
imgdistbin = imgdist.invert().
```

```
threshold(200)
```

Now you can detect them as blobs. Threshold of 200 is selected after hit-and-trial so that white region is clearly visible.

**Blobs detection.** Blobs are continuous regions of similar light-coloured pixels. Once blobs are detected, you can apply many operations on them, to find their area, length, angle, etc. You can even count them.

For example, in the following command, Minsize = 1000 indicates that you want the system to detect only those blobs which are above size 1000:

```
blobs = imgdistbin.findBlobs(minsize=1000)
```

This is also selected via hit-and-trial method such that you only get tablets area as blobs. The



Fig. 9: Image of tablets captured by the program

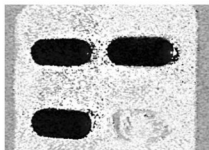


Fig. 10: Hue of the tablets



Fig. 11: Inverted image of the tablets